Ex.No: 13

Writing Wireshark Filter Expressions for Packet Capture

AIM:

To learn how to write and apply Wireshark filter expressions — both capture filters and display filters — so that only relevant packets are captured or shown during network analysis.

SOFTWARE REQUIRED:

• Computer with an active network interface (Ethernet or Wi-Fi). • Wireshark (version 3.x or later) installed with WinPcap/Npcap (Windows) • Administrator/root privileges (packet capture needs elevated rights).

THEORY:

Wireshark is an open-source protocol analyser that captures live traffic from a network interface and lets you examine each packet field by field. Because a busy network produces thousands of packets per second, filters are essential to isolate the traffic you actually care about. Wireshark supports two distinct kinds of filters, and confusing them is the most common mistake made by beginners.

Capture filters vs. Display filters Aspect Capture Filter Display Filter When applied Before capture starts After packets are captured Syntax BPF / libpcap syntax Wireshark display-filter syntax Effect Discards non-matching packets permanently Only hides packets; nothing is lost

Example tcp port 80 tcp.port == 80 Where typed Capture > Options dialog Green bar below the toolbar

A capture filter reduces the volume of data written to disk and cannot be changed once capture has begun. A display filter is applied to already-captured data, can be changed at any time, and is far more expressive because it can reach into individual protocol fields.

Display-filter comparison and logical operators Operator Alias Meaning Example == eq Equal to ip.addr == 10.0.0.1 != ne Not equal to tcp.port != 22 > gt Greater than frame.len > 1000 < lt Less than tcp.window_size < 500 && and Logical AND ip.src==10.0.0.1 && tcp.port==80 || or Logical OR dns || http ! not Logical NOT !arp

PROCEDURE:

1. Launch Wireshark. The welcome screen lists all available network interfaces along with a live sparkline of activity. 2. To use a capture filter, go to Capture → Options. In the “Capture filter for selected interfaces” box, type an expression such as host 192.168.1.10. A green box means the syntax is valid; red means it is invalid. 3. Select the interface you want (e.g., Wi-Fi or Ethernet) and click Start. Only packets matching the capture filter are recorded. 4. Generate some traffic — open a web page, run a ping, or send an email — so there is data to observe. 5. To narrow what is shown, type a display filter into the green filter bar at the top (for example http.request) and press Enter. 6. Click any packet to inspect it: the middle pane shows the decoded protocol tree and the bottom pane shows the raw bytes in hex and ASCII. 7. Click the “X” in the filter bar to clear the display filter

and see all captured packets again. 8. Stop the capture with the red square button and, if required, save the capture as a .pcapng file via File → Save As.

Worked Examples of Filter Expressions

(a) Capture filters (typed in Capture > Options) host 192.168.1.10 # all traffic to/from this host net 192.168.1.0/24 # entire subnet src host 192.168.1.10 # only packets FROM this host dst port 53 # traffic going TO port 53 (DNS) tcp port 443 # only HTTPS/TLS traffic udp # only UDP packets not arp and not broadcast # ignore ARP and broadcasts tcp port 80 and host 192.168.1.10 # combined condition

(b) Display filters (typed in the green filter bar) ip.addr == 192.168.1.10 # any packet with this IP (src or dst) ip.src == 192.168.1.10 # only where this IP is the source tcp.port == 80 # HTTP over TCP http # only HTTP protocol packets dns # only DNS packets http.request.method == "GET" # only HTTP GET requests tcp.flags.syn == 1 && tcp.flags.ack == 0 # TCP connection starts (SYN) tcp.analysis.retransmission # retransmitted TCP segments ip.addr == 192.168.1.10 && tcp.port == 443 # host AND port together frame contains "password" # packets whose bytes contain the text

SAMPLE OBSERVATION TABLE:

Filter Applied Purpose Result Observed dns Isolate name-resolution traffic Only UDP/port-53 query & response pairs listed

tcp.flags.syn==1 && tcp.flags.ack==0

Find new connection attempts

One packet per handshake start

http.request See outbound web requests

GET/POST lines with host & URI visible

ip.addr==8.8.8.8 Track one destination All packets to/from 8.8.8.8 only

RESULT: Thus the capture filters and display filters were written and applied successfully in Wireshark, and the difference between the two — syntax, timing, and effect — was verified through the observed output.

Ex.No: 14

Analysis of RTP Packet Delay and Loss using Wireshark

AIM:

To capture Real-time Transport Protocol (RTP) traffic from a VoIP/streaming session and analyse packet delay (jitter), packet loss and out-of-order delivery using the telephony analysis tools built into Wireshark.

SOFTWARE REQUIRED:

• Computer with an active network interface (Ethernet or Wi-Fi). • Wireshark (version 3.x or later) installed with WinPcap/Npcap (Windows) • Administrator/root privileges (packet capture needs elevated rights).

THEORY:

RTP (Real-time Transport Protocol) carries audio and video for VoIP and streaming applications. It runs over UDP because retransmitting late media is useless — a delayed voice sample is better dropped than replayed. Each RTP packet carries a sequence number and a timestamp, which together allow the receiver (and Wireshark) to detect the three quality problems that matter for real-time media: • Packet loss — a gap in the RTP sequence numbers means one or more packets never arrived. • Jitter (delay variation) — the variation in the inter-arrival time of packets. Even if average delay is fine, high jitter causes choppy audio because packets arrive unevenly. • Out-of-order / late packets — packets arriving after the jitter buffer has already played their slot are effectively lost.

RTP is usually accompanied by RTCP (RTP Control Protocol), which periodically reports statistics such as cumulative loss and jitter between the endpoints. Wireshark understands both.

Key metrics Wireshark reports

Metric Meaning Acceptable value (voice) Max / Mean Jitter Variation in packet arrival time < 30 ms mean is good Lost packets (%) Sequence-number gaps as a fraction < 1 % is good; > 5 % is poor Delta (ms) Time since previous packet in the stream ~20 ms for G.711 at 50 pkt/s Skew Drift between expected & actual arrival Near 0 is ideal

RTP Stream Jitter and Packet loss

PROCEDURE:

1. Start Wireshark and begin capturing on the interface that will carry the call. 2. Place a VoIP call between the two endpoints (or open an existing VoIP .pcap file via File → Open). 3. Let the call run for at least 20–30 seconds so enough media flows, then stop the capture. 4. If the media is not automatically recognised as RTP, right-click a UDP packet from the media stream, choose Decode As…, and set the protocol to RTP. 5. Open Telephony → RTP → RTP Streams. Wireshark lists every

RTP stream with source/destination addresses, payload codec, packet count, lost packets and maximum jitter. 6. Select the stream of interest and click Analyze. The per-packet analysis window opens. 7. Study the Analyze window columns — Sequence number, Delta (ms), Jitter (ms), Skew and Status. Any row marked with a warning (e.g., —Wrong sequence number‖ or —Incorrect timestamp‖) indicates loss or reordering. 8. Read off the summary line at the bottom: total packets, lost packets and percentage, max jitter and mean jitter. 9. For a live quality feel, click Play Streams to hear the reconstructed audio, or use Telephony → VoIP Calls for a call-level view and a flow graph. 10. Optionally view control statistics with the display filter rtcp, which shows the loss and jitter each endpoint reports.

Telephony ->RTP->RTP Streams Window

Useful Display Filters rtp # show only RTP media packets rtcp # show only RTP control/report packets rtp.ssrc == 0x12345678 # a single media stream by its SSRC id rtp.marker == 1 # talk-spurt / frame boundary markers sip # the SIP signalling that set up the call

SAMPLE OBSERVATION TABLE:

Fill in the values read from the RTP Streams and Analyze windows for your own capture:

Parameter Stream A → B Stream B → A

Codec / Payload G.711 (PCMU) G.711 (PCMU)

Total packets 1500 1498

Lost packets (count / %) 6 / 0.40 % 2 / 0.13 %

Max jitter (ms) 18.4 12.1

Mean jitter (ms) 6.2 4.9

Max delta (ms) 42.7 31.5

Quality assessment Good Good

RESULT: Thus an RTP stream was captured and analysed in Wireshark. Packet delay (jitter), packet loss and out-of-order delivery were measured using the RTP Streams.

Ex.No: 15

Use Wireshark to Inspect Packets on Your Network

AIM:

To capture live traffic from your own network interface and inspect the packets — examining protocol layers, following a TCP conversation, and reading traffic statistics — in order to understand how data is exchanged on the network.

SOFTWARE REQUIRED:

 A computer connected to a live network (Ethernet or Wi-Fi). Wireshark installed with Npcap/libpcap and administrator privileges. A web browser or ping utility to generate observable traffic.

THEORY:

Every action on a network — loading a page, resolving a domain, sending a message — is broken into packets that travel between hosts. Wireshark places the network card into a mode where it can read these packets and decode them according to the TCP/IP model. Understanding a single packet means understanding the layered structure wrapped around the data:

Layer Protocol example What Wireshark shows Physical/Data-Link Ethernet II Source & destination MAC addresses Network IPv4 / IPv6 Source & destination IP addresses, TTL Transport TCP / UDP Port numbers, sequence & ack numbers, flags Application HTTP, DNS, TLS Requests, responses, queries, payload

Wireshark's main window has three panes: the packet list (one line per packet), the packet details tree (the decoded layers of the selected packet), and the packet bytes pane (the same data in raw hex/ASCII).

PROCEDURE:

1. Open Wireshark. On the welcome screen, double-click the active interface (the one whose sparkline is moving) to begin capturing. 2. Generate traffic: open a website, or from a terminal run a ping so you have known packets to find. 3. ping www.google.com nslookup www.google.com 4. Watch the packet list fill in real time. Stop the capture (red square) after a few seconds so the list stops moving and is easier to read. 5. Apply a display filter to focus, e.g. type dns to see the name lookups, or icmp to see your ping echo requests and replies. 6. Click one packet and expand the details tree in the middle pane. Read down through Ethernet → IP → TCP/UDP → application layers and note the addresses and ports. 7. Right-click a TCP packet and choose Follow → TCP Stream to reassemble an entire conversation (for example the full HTTP request and response) into a single readable window. 8. Open Statistics → Protocol Hierarchy to see the percentage of traffic by protocol — a quick profile of what your machine is doing. 9. Open Statistics → Conversations to list every pair of hosts talking, with byte and packet counts, and Statistics → Endpoints to see the busiest addresses. 10. Optionally open Statistics → I/O Graph to plot traffic volume over time. 11. Save the capture via File → Save As if you need to submit it or analyse it later.

SAMPLE OBSERVATION TABLE:

Packet No.

Source IP Destination IP Protocol Info

1 192.168.1.5 192.168.1.1 DNS Standard query A www.google.com 2 192.168.1.1 192.168.1.5 DNS Response A 142.250.x.x

3 192.168.1.5 142.250.x.x TCP SYN → connection request 4 142.250.x.x 192.168.1.5 TCP SYN, ACK → accepted

5 192.168.1.5 142.250.x.x ICMP Echo (ping) request

RESULT:

Thus, the live packets were captured from the network interface and inspected in Wireshark.

Ex.No: 16

Create a Simple Network using Cisco Packet Tracer

AIM:

To design and build a simple Local Area Network (LAN) in Cisco Packet Tracer by connecting end devices through a switch, assigning IP addresses, and verifying end-to-end connectivity with the ping command.

SOFTWARE REQUIRED:

 A computer connected to a live network (Ethernet or Wi-Fi). Cisco Packet Tracer (version 8.x recommended Basic knowledge of IPv4 addressing and subnet masks.

THEORY:

Cisco Packet Tracer is a network simulation tool that lets you build and test networks without physical hardware. A simple LAN connects several end devices (PCs) to a central switch. A switch forwards frames using MAC addresses and creates one broadcast domain, so all devices attached to it — if given IP addresses in the same subnet — can communicate directly. In this experiment three PCs are joined to a single switch on the network 192.168.1.0/24 and connectivity is proven with ping, which sends ICMP echo requests and waits for replies.

Addressing plan

Device Interface IP Address Subnet Mask

PC0 FastEthernet0 192.168.1.1 255.255.255.0

PC1 FastEthernet0 192.168.1.2 255.255.255.0

PC2 FastEthernet0 192.168.1.3 255.255.255.0

PROCEDURE:

1. Open Cisco Packet Tracer. You are presented with a blank logical workspace (the large white area). 2. Add a switch: from the bottom-left device box choose Network Devices → Switches, pick the 2960 model, and click on the workspace to drop it. 3. Add three PCs: choose End Devices, select PC, and place PC0, PC1 and PC2 around the switch. 4. Connect the devices: choose the Connections (lightning-bolt) icon and pick Copper Straight-Through cable. Click a PC, choose its FastEthernet0 port, then click the switch and choose a free FastEthernet port. Repeat for all three PCs. 5. Wait until the link lights turn green (a few seconds), indicating the ports are up. 6. Configure IP addresses: click PC0 → Desktop tab → IP Configuration. Select Static and enter IP 192.168.1.1 and subnet mask 255.255.255.0. Close the window. 7. Repeat step 6 for PC1 (192.168.1.2) and PC2 (192.168.1.3), all with mask 255.255.255.0. 8. Test connectivity: on PC0 → Desktop → Command Prompt, run the ping command to another PC. 9. PC> ping 192.168.1.2 PC> ping 192.168.1.3 10. A successful ping shows four replies with round-trip times. Repeat from other PCs to confirm every device can reach every

other device. 11. Optional: switch to Simulation mode (bottom-right) and re-run a ping to watch the ICMP packet animate hop-by-hop from source to switch to destination. 12. Save the file via File → Save As with a .pkt extension.

SAMPLE OBSERVATION TABLE:

C:\> ping 192.168.1.2

Pinging 192.168.1.2 with 32 bytes of data: Reply from 192.168.1.2: bytes=32 time<1ms TTL=128 Reply from 192.168.1.2: bytes=32 time<1ms TTL=128 Reply from 192.168.1.2: bytes=32 time<1ms TTL=128 Reply from 192.168.1.2: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.2: Packets: Sent = 4, Received = 4, Lost = 0 (0% loss), If the first request “times out”, it is normal — it is caused by ARP resolving the MAC address; subsequent replies confirm the link is working.

RESULT: Thus a simple LAN of three PCs connected through a 2960 switch was built in Cisco Packet Tracer. IP addresses were assigned in the 192.168.1.0/24 network, and end-to-end connectivity between all devices was successfully verified using the ping command.

Ex.No: 17

Study of Topology - Benchmark LAN Trainer Kit

AIM:

To study the various network topologies - bus, star, ring, mesh, tree and hybrid- and to physically construct and observe them using the Benchmark LAN Trainer Kit.

SOFTWARE REQUIRED:

• Benchmark LAN Trainer Kit with its node modules, central hub/switch module and patch cords. • Set of connecting cables (RJ-45 patch cords or the kit's supplied connectors). • Kit power supply and the accompanying observation/indicator panel.

THEORY:

Network topology is the physical or logical arrangement in which nodes (computers, terminals or the kit's node modules) are interconnected. The topology determines how data flows, how much cabling is needed, how the network behaves when a link or node fails, and how easy it is to expand. A LAN trainer kit provides node modules and a central connecting module so that each topology can be wired up by hand and its behaviour — signal path, fault effect, expandability — directly observed on the kit's indicator panel.

Addressing plan

Topology Arrangement Advantages Disadvantages

Bus All nodes share one common backbone cable with terminators at both ends.

Simple; least cabling; cheap.

One cable fault stops the whole network; hard to troubleshoot.

Star Every node connects to a central hub/switch.

Easy to add/remove nodes; single node fault does not affect others.

Central device failure downs the whole network; more cable.

Ring Nodes form a closed loop; data passes node to node.

Orderly, predictable data flow; no collisions with token passing.

A single break can halt the ring; adding nodes disrupts it.

Mesh Every node links directly to every other node.

Highly reliable; multiple paths; faulttolerant.

Very expensive; huge amount of cabling.

Tree Groups of star networks joined to a bus backbone (hierarchy).

Scalable; good for large structured networks.

Backbone failure affects large sections; complex.

Hybrid Combination of two or more of the above.

Flexible; tailored to needs.

Design and management are complex and costly.

PROCEDURE:

1. Place the LAN Trainer Kit on the bench and connect its power supply. Switch it on and confirm the indicator lamps/LEDs are functioning. 2. Identify the components: the individual node modules, the central hub/switch module, the backbone connector, and the supplied patch cords.

Bus topology 3. Connect all node modules along the single common backbone line provided on the kit, fitting the terminators at both ends of the bus. 4. Send data from one node and observe on the indicator panel that the signal travels along the shared backbone and is seen by all nodes; only the addressed node accepts it. 5. Disconnect the backbone at a mid-point and note that communication beyond the break stops — demonstrating the bus's single point of failure.

Star topology 6. Connect each node module by a separate patch cord to a port on the central hub/switch module. 7. Transmit from one node and observe that data passes through the central module to reach the destination node. 8. Disconnect one node's cable and confirm the other nodes continue to communicate; then power off the central module and note that the whole network stops — showing the star's dependence on the central device.

Ring topology 9. Wire the node modules in a closed loop — the output of each node to the input of the next — until the last node connects back to the first. 10. Send data and observe it circulating from node to node around the ring until it reaches the intended destination. 11. Open the loop at one link and observe that the ring's data flow is interrupted.

Mesh topology 12. Using the available patch cords, connect each node module directly to every other node module. 13. Transmit between two nodes and observe that, because multiple direct paths exist, communication continues even if one link is removed — demonstrating fault tolerance. 14. For each topology, record on the observation sheet: number of cables used, the path taken by the data, and the effect of a single link or node failure.

SAMPLE OBSERVATION TABLE:

Topology

Cables used (for n nodes)

Effect of one link failure

Ease of adding a node

Bus n + backbone Whole segment may fail Easy

Star n Only that node affected Very easy

Ring n Ring flow interrupted Difficult

Mesh n(n−1)/2 No effect (alternate paths) Difficult

RESULT:

Thus, the bus, star, ring and mesh network topologies were studied and physically constructed on the Benchmark LAN Trainer Kit.